INTERNSHIP PROJECT REPORT

PERSONAL FINANCE ADVISOR BOT

Web Application for Personal Budgeting and AI-Assisted Insights

Submitted to: NASSCOM (please confirm the official spelling/name used by your internship program)

Student: Pranjal Gaikwad

Programme / Branch: Second Year Engineering — AI / Data Science

Roll Number: 22228

Internship Dates: [Insert internship start and end dates]

Mentor / Supervisor: [Insert mentor/supervisor name]

GitHub Repository: [Insert your GitHub repository URL]

Prepared for internship documentation and review

Table of Contents

1. Declaration

2. Acknowledgement

3. Abstract

4. 1. Introduction

5. 2. Problem Statement

6. 3. Objectives

7. 4. Scope of the Project

8. 5. Technology Stack

9. 6. System Design and Working

10. 7. Main Features

11. 8. Implementation Details

12. 9. Testing and Validation

13. 10. Results and Outcome

14. 11. Challenges and Learning

15. 12. Limitations and Future Enhancements

16. 13. GitHub and Project Resources

17. 14. Conclusion

18. References

19. Submission Checklist

Declaration

I declare that this report describes the Personal Finance Advisor Bot project developed as part of my internship/project work. The report is prepared for review and submission, and any implementation details, screenshots, repository links, or test results should be verified against the final project before submission.

Acknowledgement

I would like to thank my internship program and mentors for the opportunity to work on this project. I also acknowledge the support of the development tools used during implementation, including Antigravity and GitHub, which helped with coding, project organization, and version control.

Abstract

The Personal Finance Advisor Bot is a web application designed to help users record income and expenses, organize spending by category, monitor budgets, track savings goals, and review their financial activity through a dashboard and reports. The project combines deterministic calculations in Python for financial totals with AI-generated natural-language explanations and suggestions. The intended outcome is to make everyday budgeting easier to understand and manage. The application is a planning and educational tool; it should not be treated as a substitute for professional financial advice.

1. Introduction

Managing personal finances requires consistent tracking of income, expenses, budgets, and savings. When this information is scattered across notes or spreadsheets, it can be difficult to identify spending patterns or determine whether monthly goals are being met.

The Personal Finance Advisor Bot aims to bring these activities into one interface. Users can enter financial transactions, organize them, review summaries, set budgets or goals, and receive explanatory insights that help them reflect on their spending.

2. Problem Statement

Many users find it difficult to maintain a clear picture of their monthly finances. Manual tracking can be time-consuming, and raw transaction lists do not always make it easy to understand spending trends. A simple, centralized application can help users organize financial information and turn it into understandable summaries.

3. Objectives

• Provide a convenient way to record income and expenses.

• Categorize transactions to make spending easier to review.

• Support monthly budgets and savings-goal tracking.

• Display key financial summaries through a dashboard.

• Generate readable spending insights and savings suggestions using an AI model.

• Produce monthly reports that help users review their financial activity.

4. Scope of the Project

The project focuses on personal budgeting and financial tracking. Its core scope includes user access, income and expense management, category-based summaries, budget tracking, savings progress, dashboard views, AI-assisted explanations, and monthly reporting, subject to the features present in the final build.

Investment execution, bank-account integration, credit decisions, and guaranteed financial outcomes are outside the intended scope unless separately implemented and reviewed.

5. Technology Stack

• Backend: Python with Flask.

• Database/ORM: SQLAlchemy; the exact database engine should be confirmed from the project configuration.

• Authentication: Flask-Login.

• Frontend: HTML5, CSS3, JavaScript, and the styling framework used in the final project (Tailwind CSS and/or Bootstrap 5).

• UI assets: Font Awesome and Google Fonts (Inter), if retained in the final build.

• AI: Google Gemini 2.0 Flash (gemini-2.0-flash) for natural-language insights and recommendations, if this is the model configured in the submitted version.

• Development and version control: Antigravity for AI-assisted development and GitHub for repository hosting/version history.

6. System Design and Working

1. The user signs in and enters income or expense information.

2. The backend validates incoming data and stores the relevant records in the database.

3. Python logic calculates totals, remaining budget, category summaries, and savings progress. Core financial arithmetic should be deterministic and should not depend on the language model.

4. The frontend requests and displays dashboard data using browser-side JavaScript and fetch/API requests where implemented.

5. When requested, the AI component receives suitable summarized context and returns a natural-language explanation or suggestion.

6. The user reviews the dashboard and monthly report to make their own budgeting decisions.

7. Main Features

• Income tracking: record income entries and review totals.

• Expense tracking: record spending and assign categories and dates.

• Budget management: set and monitor planned spending limits.

• Savings goals: view progress toward a target amount.

• Dashboard: review key totals and financial summaries in one place.

• AI-assisted insights: receive plain-language explanations of spending patterns and possible savings actions.

• Monthly report: review activity over a selected period.

• Authentication: provide account access controls through the implemented login system.

8. Implementation Details

The application follows a web-based architecture with a browser interface, a Flask backend, and database-backed persistence. Routes and backend functions handle application requests, while the frontend presents forms and financial summaries. SQLAlchemy is used to interact with stored records, and Flask-Login supports user-session management.

A key design principle is to separate calculations from generated text. Totals, budget balances, and progress values should be computed by application code. The AI model can explain those computed values or suggest habits, but it should not be the source of truth for numerical balances. API keys and other secrets must be kept out of public repositories and managed through environment variables or an equivalent secure configuration.

9. Testing and Validation

Testing should verify account access, creation and editing of income/expense entries, category totals, budget calculations, savings progress, dashboard updates, report generation, and error handling for invalid input.

Project notes previously described a suite of 43 automated unit/integration tests. Before submitting this report, rerun the tests in the current version and replace this sentence with the verified result, including the test command and pass/fail count. Do not claim that all tests pass unless you have confirmed it from the final repository.

10. Results and Outcome

The intended result is a single application where users can maintain financial records and understand their spending through summaries, budget indicators, savings goals, and AI-assisted explanations. Add screenshots from the final working application to demonstrate the login page, dashboard, transaction forms, budgets/goals, and monthly report.

[Insert screenshots and brief captions here. Only include screenshots from the current working version.]

11. Challenges and Learning

Potential project challenges include validating user-entered financial data, keeping dashboard totals consistent with stored transactions, designing a clear interface, handling AI responses safely, configuring the application environment, and maintaining a clean GitHub repository.

Through this work, I developed practical experience with AI-assisted development, web application structure, Python/Flask, database integration, frontend-backend communication, version control, debugging, and project documentation. Edit this section to reflect the tasks you personally completed.

12. Limitations and Future Enhancements

Possible limitations include manual transaction entry, dependence on an available AI service for AI-generated insights, and the need for careful handling of sensitive financial information.

Future improvements may include CSV import/export, richer spending charts, recurring transaction support, more accessible UI design, stronger validation and security checks, optional bank-data integrations where legally and technically appropriate, and more comprehensive testing.

13. GitHub and Project Resources

Repository URL: [Insert your GitHub repository URL]

Project folder: personal-finance-advisor (confirm the actual repository/folder name).

Before sharing the repository, check that it contains a README with setup instructions, a requirements file, environment-variable guidance, and no API keys, passwords, personal data, or other secrets.

14. Conclusion

The Personal Finance Advisor Bot demonstrates how a web application can combine structured financial tracking with AI-assisted explanations. By keeping financial calculations in application code and using AI for supporting natural-language insights, the project aims to provide a more understandable budgeting experience. Further validation, security review, and user feedback can improve reliability and usability.

References

• Flask documentation: https://flask.palletsprojects.com/

• SQLAlchemy documentation: https://docs.sqlalchemy.org/

• Flask-Login documentation: https://flask-login.readthedocs.io/

• Google Gemini API documentation: https://ai.google.dev/gemini-api/docs

• GitHub documentation: https://docs.github.com/

Use only references relevant to the technologies actually present in your final build.

Submission Checklist

■ Confirm the official internship/company name and required report format.

■ Add internship dates and mentor/supervisor details.

■ Insert the correct GitHub repository URL.

■ Add screenshots from the final working app.

■ Verify the actual tech stack and database from the repository.

■ Rerun tests and update testing results accurately.

■ Remove all placeholders and checklist instructions before final submission.

■ Confirm whether the internship portal also requires a weekly log, completion certificate, or separate README/project documentation.